SUPFile

Plateforme de Stockage Cloud Collaboratif

Projet 4PROJ — SUPINFO

Documentation Technique Complète

Équipe : Abdoulaye Fofana, Rayelen Guesmi, Tharshan Sivapalan, Kendall Gbaguidi

Promotion : PGE.4 — SUPINFO

Année : 2025–2026

URL Web : https://www.supfile-cloud.fr

URL API : https://api.supfile-cloud.fr/api

Dépôt Git : https://github.com/Abdoulayefoff/supfile-project

Version : 1.0

Date : 24 mai 2026

© 2025–2026 SUPFile SUPINFO — Tous droits réservés

Table des matières

1 Introduction et Vision du Projet **4**

1.1 Présentation de SUPFile 4

1.2 Objectifs du Projet 5

1.3 Équipe de Développement 5

2 Procédure d'Installation et Prérequis **6**

2.1 Prérequis Système 6

2.1.1 Ressources Minimales Recommandées 6

2.2 Variables d'Environnement Locales 6

2.3 Installation Rapide (One-Command Setup) 7

2.3.1 Ce que le script fait automatiquement 7

2.4 Commandes Make Disponibles 8

3 Architecture Générale du Système **9**

3.1 Vue d'Ensemble Architecturale 9

3.2 Architecture en Couches (Backend) 9

3.3 Pattern Stockage : Strategy / Factory 10

3.3.1 Configuration des Limites d'Upload 10

4 Diagrammes UML **12**

4.1 Diagramme de Cas d'Utilisation 12

4.2 Modèle de Données (Schéma Relationnel) 13

4.2.1 Relations entre les tables 13

4.2.2 Table users 14

4.2.3 Table folders 14

4.2.4 Table files 14

4.2.5 Table shares 15

4.2.6 Table shared_folders 15

4.3 Diagramme de Séquence — Upload de Fichier 15

4.4 Diagramme de Séquence — Accès à un Partage Public 16

5 Architecture de l'API REST **17**

5.1 Conventions Générales 17

5.2 Codes d'Erreur API 17

5.3 Authentification (`/api/auth`) 17

5.3.1 Principe JWT 17

5.3.2 Niveaux de sécurité . . . 17
5.3.3 Routes OAuth2 . . . 18
5.4 Gestion des Fichiers (`/api/files`) . . . 18
5.4.1 Concepts clés . . . 18
5.5 Gestion des Dossiers (`/api/folders`) . . . 19
5.5.1 Hiérarchie . . . 19
5.6 Corbeille (`/api/trash`) . . . 19
5.7 Partage (`/api/shares`) . . . 19
5.7.1 Types de partage . . . 19
5.8 Tableau de Bord (`/api/dashboard`) . . . 20
5.9 Gestion du Compte (`/api/user`) . . . 20
5.10 Pages Légales (`/api/legal`) — Accès Public . . . 20

6 Sécurité de l'Application 21
6.1 Protection contre les Injections SQL . . . 21
6.2 Protection Path Traversal . . . 21
6.3 Rate Limiting — Mot de Passe Oublié . . . 21
6.4 Sécurité Mobile — expo-secure-store . . . 21
6.5 OAuth 2.0 — WebView Isolée (Mobile) . . . 21
6.6 Conformité OWASP Top 10 . . . 22

7 Justification des Choix Technologiques 23
7.1 Backend : Symfony 7.4 LTS / PHP 8.3 . . . 23
7.2 Frontend Web : React 19 + TypeScript + Vite . . . 23
7.3 Frontend Mobile : React Native + Expo SDK 54 . . . 23

8 Guide de Déploiement 25
8.1 Déploiement Local (Docker Compose) — Architecture Principale . . . 25
8.2 Déploiement Prod : Microsoft Azure — Bonus . . . 25
8.2.1 Variables d'environnement de Production . . . 26

9 Pipeline CI/CD 27
9.1 Vue d'Ensemble . . . 27

10 Dépannage Technique (Troubleshooting) 28
10.1 Problèmes Courants et Solutions . . . 28
10.1.1 Le conteneur `api` ne démarre pas . . . 28
10.1.2 Erreur de connexion MySQL au démarrage . . . 28
10.1.3 Migrations Doctrine échouent . . . 28
10.1.4 Le mobile ne peut pas joindre l'API locale . . . 29
10.1.5 Erreur OAuth — Callback refusé . . . 29

10.1.6 Upload refusé — HTTP 413 ou quota dépassé . . . 29
10.1.7 JWT expiré — HTTP 401 après 1 heure . . . 29
10.1.8 Consulter les Logs . . . 29

11 Charte Graphique **30**
11.1 Palette de Couleurs . . . 30
11.2 Gradient Principal . . . 30
11.3 Typographie . . . 30
11.4 Thèmes Clair et Sombre . . . 30
11.5 Règles de Design . . . 31

12 Fonctionnalités et Qualité du Code **32**
12.1 Fonctionnalités Implémentées . . . 32
12.2 Standards de Qualité . . . 33
12.3 Stratégie de Tests . . . 33
12.3.1 Backend — Tests PHP (PHPUnit 12) . . . 33
12.3.2 Frontend Web — Tests TypeScript (Jest) . . . 34
12.3.3 Frontend Mobile — Tests TypeScript (Jest + Testing Library) . . . 34

13 Conclusion **36**
13.1 Bilan . . . 36
13.2 Points Forts Techniques . . . 36
13.3 Perspectives d'Évolution . . . 36

Références **37**

Annexe — Processus de Développement **38**

Chapitre 1

Introduction et Vision du Projet

1.1 Présentation de SUPFile

SUPFile est une plateforme de stockage cloud collaborative inspirée de Dropbox et Google Drive, développée dans le cadre du projet 4PROJ à SUPINFO. Elle permet à ses utilisateurs de stocker, organiser, prévisualiser et partager des fichiers numériques via une interface web moderne et une application mobile native.

Architecture centrale

L'ensemble du projet s'exécute localement via un `docker-compose.yml` orchestrant 3 services : backend Symfony (`api`), base de données MySQL (`database`) et frontend web (`web`). Les fichiers physiques sont stockés sur un volume Docker `storage_data` monté dans le conteneur `api`. Les métadonnées (nom, taille, type MIME, chemin) sont persistées en base MySQL via Doctrine ORM. Le frontend mobile (React Native + Expo) n'est **pas dockerisé** : il s'exécute via `npx expo start` et se connecte à l'API locale.

En complément, le projet a été déployé en production sur **Microsoft Azure** (App Service + Azure MySQL Flexible + Azure Blob Storage) pour démontrer la scalabilité de l'architecture.

L'application répond aux exigences fonctionnelles suivantes :

— Authentification sécurisée par JWT RS256 avec support OAuth 2.0 (Google, GitHub, Microsoft)

— Gestion complète de fichiers et dossiers (upload jusqu'à **2 Go**, téléchargement, suppression douce, déplacement, renommage)

— Corbeille (*soft delete*) — restauration ou suppression définitive

— Quota utilisateur de **30 Go** par compte (`quota_used` / `quota_limit`)

— Prévisualisation en ligne (images, PDF, vidéos, audio)

— Partage de fichiers et dossiers par liens publics, optionnellement protégés par mot de passe et/ou date d'expiration

— Tableau de bord analytique avec recherche globale

— Application mobile iOS/Android (React Native + Expo SDK 54)

— Déploiement production sur Microsoft Azure (*bonus*)

1.2 Objectifs du Projet

1. Mettre en œuvre une architecture locale conteneurisée (Docker Compose) pour le développement et la livraison
2. Concevoir une API REST robuste, scalable et sécurisée
3. Respecter les bonnes pratiques OWASP (injection, broken auth, IDOR, etc.)
4. Livrer deux clients fonctionnels : web (React) et mobile (React Native + Expo)
5. **Bonus** — Implémenter une CI/CD automatisée (GitHub Actions)
6. **Bonus** — Déployer en production sur Azure avec certificat SSL et CDN Cloudflare

1.3 Équipe de Développement

Membre	Rôle principal	Périmètre
Abdoulaye FOFANA	Backend Lead	API Symfony, Azure, CI/CD, Tests
Rayelen GUESMI	Frontend Mobile	Application React Native, Tests
Tharshan SIVAPALAN	Frontend Web	Interface React, Tests
Kendall GBAGUIDI	Backend	Modèle BDD, Doctrine ORM

Chapitre 2

Procédure d'Installation et Prérequis

2.1 Prérequis Système

Outil	Version min.	Usage
Docker	24+	Conteneurisation de tous les services
Docker Compose	2.20+	Orchestration locale multi-conteneurs
PHP	8.2+	Runtime backend (image Docker `api` en 8.3)
Composer	2.7+	Gestionnaire de dépendances PHP
Node.js	22+	Build frontend web et outillage Expo
npm	10+	Gestionnaire de paquets JavaScript
Git	2.40+	Gestion du code source
OpenSSL	3.x	Génération des secrets et clés JWT
EAS CLI	12+	Build APK/IPA Expo (mobile uniquement)

2.1.1 Ressources Minimales Recommandées

— RAM : 4 Go minimum (8 Go recommandés pour tous les conteneurs)

— CPU : 2 cœurs minimum

— Disque : 5 Go libres (images Docker + dépendances)

— OS : Linux, macOS ou Windows (WSL2 fortement recommandé)

— Ports libres : `8000` (backend), `3000` (frontend web), `3307` (MySQL, port hôte → conteneur)

2.2 Variables d'Environnement Locales

Le fichier `.env` est généré automatiquement par `start.sh`. Pour une configuration manuelle, voici les variables à définir :

Variable	Exemple	Description
APP_ENV	dev	Environnement Symfony
APP_SECRET	(32 hex)	Secret CSRF Symfony
DATABASE_URL	voir ci-dessous	DSN MySQL complet
DB_NAME	supfile_db	Nom de la base
DB_USER	supfile	Utilisateur MySQL
DB_PASSWORD	(généré)	Mot de passe MySQL
DB_ROOT_PASSWORD	(généré)	Root MySQL
JWT_SECRET_KEY	(clé RSA privée)	Signature JWT
JWT_PUBLIC_KEY	(clé RSA publique)	Vérification JWT
JWT_PASSPHRASE	(généré)	Passphrase clé privée
STORAGE_DRIVER	local	local ou azure
STORAGE_PATH	/var/www/html/storage	Répertoire fichiers
GOOGLE_CLIENT_ID	(OAuth console)	App Google OAuth
GOOGLE_CLIENT_SECRET	(OAuth console)	Secret Google OAuth
GITHUB_CLIENT_ID	(OAuth GitHub)	App GitHub OAuth
GITHUB_CLIENT_SECRET	(OAuth GitHub)	Secret GitHub OAuth
MICROSOFT_CLIENT_ID	(Azure AD)	App Microsoft OAuth
MICROSOFT_CLIENT_SECRET	(Azure AD)	Secret Microsoft OAuth
MAILER_DSN	smtp://...	SMTP reset mot de passe
CORS_ALLOW_ORIGIN	http://localhost:3000	Frontend web autorisé

2.3 Installation Rapide (One-Command Setup)

```
git clone git@github.com:Abdoulayefoff/supfile-project.git
cd supfile-project
chmod +x start.sh
./start.sh
```

Listing 2.1 – Installation complète en une commande

Pourquoi un script start.sh ?

Le sujet 4PROJ interdit les secrets en clair dans le code. Le script `start.sh` génère automatiquement le fichier `.env` avec des secrets cryptographiquement sûrs via `openssl rand -hex 32` — garantissant la sécurité par défaut (*Security by Default*).

2.3.1 Ce que le script fait automatiquement

1. Vérifie la présence de Docker et Docker Compose
2. Génère le fichier `.env` avec des secrets via `openssl rand -hex 32`
3. Installe les dépendances Composer (répertoire `vendor/`)

4. Génère la paire de clés RSA pour JWT (LexikJWT)
5. Démarre tous les conteneurs Docker en arrière-plan
6. Crée la base de données MySQL et l'utilisateur applicatif
7. Exécute les migrations Doctrine
8. Installe les dépendances frontend (`npm install`)
9. Effectue des vérifications de santé (*health checks*) sur chaque service
10. Affiche les URLs d'accès avec un résumé coloré

2.4 Commandes Make Disponibles

Commande	Description
`make install`	Installation complète du projet
`make start`	Démarre tous les conteneurs
`make stop`	Arrête tous les conteneurs
`make restart`	Redémarre les conteneurs
`make rebuild`	Reconstruit les images Docker (sans cache)
`make test`	Lance tous les tests (backend + frontend)
`make test-coverage`	Tests backend avec rapport de couverture HTML
`make logs`	Affiche les logs de tous les conteneurs

Chapitre 3

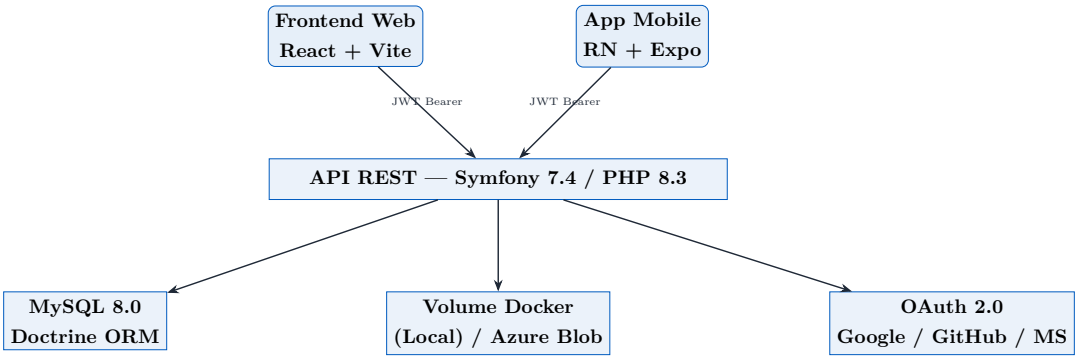
Architecture Générale du Système

3.1 Vue d'Ensemble Architecturale

SUPFile adopte une architecture **trois-tiers orientée API** : deux clients indépendants (web et mobile) consomment une API REST centralisée, elle-même connectée à une base de données MySQL et un stockage binaire de fichiers.

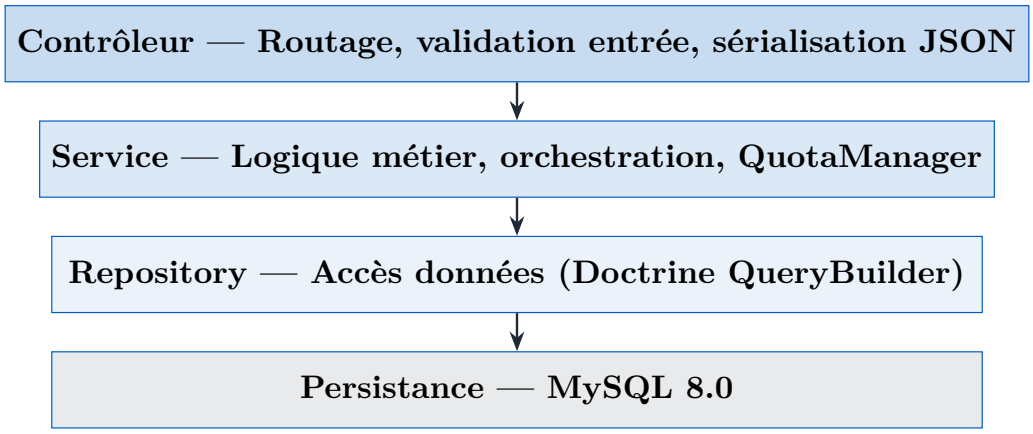
Stockage — local vs production

- **Local (Docker Compose)** — fichiers physiques sur le volume Docker `storage_data` monté dans `/var/www/html/storage` ; métadonnées en base MySQL via Doctrine ORM.
- **Production (Azure)** — fichiers physiques dans **Azure Blob Storage** (LRS Standard) ; métadonnées dans **Azure MySQL Flexible Server**. La variable `STORAGE_DRIVER=azure` bascule automatiquement l'implémentation via `StorageFactory`.
- **Mobile** — React Native + Expo n'est **pas dockerisé**. Il se lance via `npx expo start` et se connecte à l'API (locale ou en production).



3.2 Architecture en Couches (Backend)

Le backend Symfony respecte une architecture strictement en couches :



Séparation des responsabilités

Les contrôleurs ne contiennent **aucune logique métier**. Toute vérification de quota, toute opération sur les fichiers, toute gestion des droits d'accès est déléguée aux Services. Les Repositories encapsulent exclusivement les requêtes Doctrine.

3.3 Pattern Stockage : Strategy / Factory

Le stockage des fichiers est abstrait derrière une interface, permettant de basculer entre stockage local et Azure Blob sans modifier le code métier :

```
interface FileStorageInterface {
    public function storeFile(UploadedFile $uploadedFile, User $owner):
        string;
    public function getFilePath(File $file): string;
    public function deleteFile(File $file): bool;
    public function exists(File $file): bool;
    public function getStream(File $file): mixed;
}
// StorageFactory::create() retourne LocalFileStorage ou AzureBlobStorage
// selon STORAGE_DRIVER=local|azure dans .env
```

Listing 3.1 – Interface FileStorageInterface (`backend/src/Service/Storage/FileStorageInterface.php`)

3.3.1 Configuration des Limites d'Upload

Les tailles maximales de téléversement (upload) sont strictement définies pour garantir la stabilité du système. Elles se divisent en deux catégories :

— **Fichiers généraux (Documents, Médias, etc.) :** Limite fixée à **2 Go**.
— **Avatars de profil :** Limite fixée à **2 Mo**.

Modifier la limite des Fichiers généraux (ex : passage à 5 Go)

Pour augmenter cette limite, il est impératif de modifier de manière coordonnée trois composants :

1. **Nginx (Reverse Proxy) :** Dans `backend/docker/nginx.conf`, modifier la directive `client_max_body_size 5G;`.
2. **PHP Local (Docker) :** Dans `backend/Dockerfile`, ajuster les valeurs `upload_max_filesize = 5G` et `post_max_size = 5G`.
3. **PHP Production (Azure) :** Dans `backend/Dockerfile.prod`, ajuster ces mêmes valeurs PHP.

Attention aux très gros fichiers

Augmenter significativement cette limite (par ex. 10 Go) peut entraîner des erreurs de "Timeout" (le délai de réponse du serveur expire). Il sera alors nécessaire d'ajuster le `max_execution_time` de PHP et le `proxy_read_timeout` de Nginx, ou de passer à un système d'upload par morceaux (Chunked Uploads) côté front-end. N'oubliez pas non plus que la taille est limitée in fine par le **Quota utilisateur** (fixé par défaut à 30 Go).

Modifier la limite des Avatars de profil

Cette vérification est purement applicative. Dans le fichier `backend/src/Controller/Api/UserController.php` (autour de la ligne 170), il suffit d'ajuster la condition : `if ($file->getSize() > 2 * 1024 * 1024)`.

Chapitre 4

Diagrammes UML

4.1 Diagramme de Cas d'Utilisation

Ce diagramme représente l'ensemble des actions disponibles pour chaque acteur du système. Deux acteurs principaux sont identifiés : l'**Utilisateur authentifié** (titulaire d'un compte) et le **Visiteur** (accès public aux liens de partage). Un acteur externe, le **Fournisseur OAuth**, intervient lors de la connexion sociale (Google, GitHub, Microsoft).

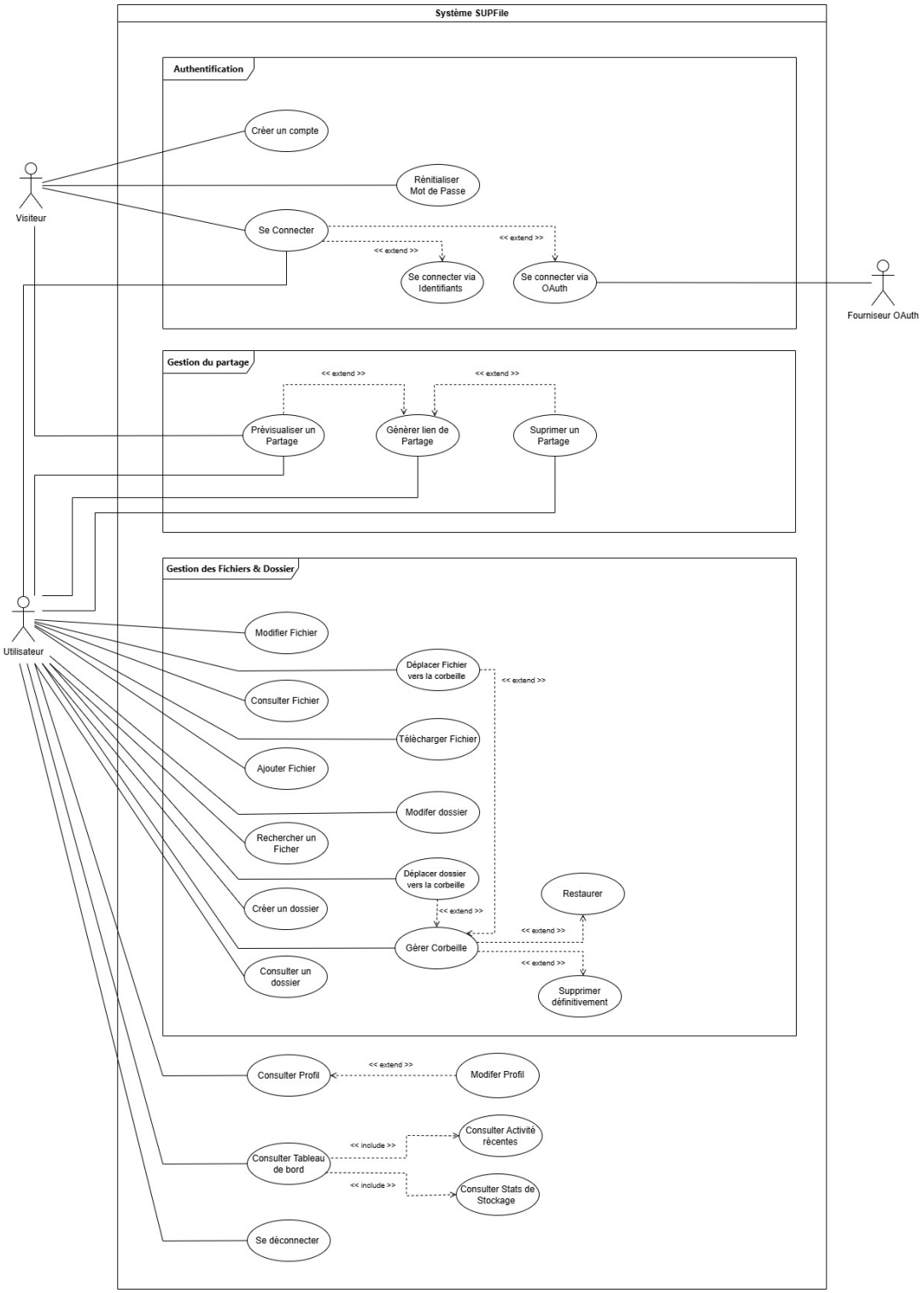


FIGURE 4.1 – Diagramme de cas d'utilisation SUPFile — vue complète

4.2 Modèle de Données (Schéma Relationnel)

4.2.1 Relations entre les tables

— **users** (1 :N) → **folders** : un utilisateur possède plusieurs dossiers

— **users** (1 :N) → **files** : un utilisateur possède plusieurs fichiers

— **folders** (1 :N) → **folders** : auto-référence (arborescence)

— **folders** (1 :N) → **files** : un dossier contient plusieurs fichiers

— **files** (1 :N) → **shares** : un fichier peut avoir plusieurs partages

— **folders** (1 :N) → **shares** : un dossier peut avoir plusieurs partages

— **folders** (N :M) → **users** via **shared_folders** : partage privé

4.2.2 Table users

Table 4.1: Structure de la table `users`

Colonne	Type	Contraintes	Description
id	INT	PK, AUTO_INCREMENT	Identifiant unique
email	VARCHAR(180)	UNIQUE, NOT NULL	E-mail de connexion
password	VARCHAR(255)	NULLABLE	Hash bcrypt (null si OAuth)
roles	JSON	NOT NULL	Rôles Symfony [`ROLE_USER`]
oauth_provider	VARCHAR(50)	NULLABLE	`google`, `github`, `microsoft`
oauth_id	VARCHAR(255)	NULLABLE	Identifiant externe OAuth
avatar	VARCHAR(255)	NULLABLE	Chemin vers l'avatar
quota_used	BIGINT	DEFAULT 0	Octets utilisés
quota_limit	BIGINT	DEFAULT 32212254720	30 Go en octets
created_at	DATETIME	NOT NULL	Date de création
updated_at	DATETIME	NOT NULL	Date de dernière modification

4.2.3 Table folders

Table 4.2: Structure de la table `folders`

Colonne	Type	Contraintes	Description
id	INT	PK, AUTO_INCREMENT	Identifiant unique
name	VARCHAR(255)	NOT NULL	Nom du dossier
owner_id	INT	FK users.id, NOT NULL	Propriétaire
parent_id	INT	FK folders.id, NULLABLE	Dossier parent (NULL = racine)
created_at	DATETIME	NOT NULL	Date de création
updated_at	DATETIME	NOT NULL	Date de modification

Index : UNIQUE sur `(owner_id, parent_id, name)` — empêche deux dossiers de même nom au même niveau.

4.2.4 Table files

Table 4.3: Structure de la table `files`

Colonne	Type	Contraintes	Description
id	INT	PK, AUTO_INCREMENT	Identifiant unique

Suite à la page suivante

Colonne	Type	Contraintes	Description
name	VARCHAR(255)	NOT NULL	Nom du fichier
owner_id	INT	FK users.id, NOT NULL	Propriétaire
folder_id	INT	FK folders.id, NULLABLE	Dossier (NULL = racine)
size	BIGINT	NOT NULL	Taille en octets
mime_type	VARCHAR(100)	NOT NULL	Type MIME
storage_path	VARCHAR(255)	NOT NULL	Chemin physique unique
is_deleted	BOOLEAN	DEFAULT 0	Corbeille (soft delete)
deleted_at	DATETIME	NULLABLE	Date de suppression
created_at	DATETIME	NOT NULL	Date de création
updated_at	DATETIME	NOT NULL	Date de modification

4.2.5 Table shares

TABLE 4.4: Structure de la table `shares`

Colonne	Type	Contraintes	Description
id	INT	PK, AUTO_INCREMENT	Identifiant unique
token	VARCHAR(64)	UNIQUE, NOT NULL	Token URL unique
file_id	INT	FK files.id, NULLABLE	Si partage fichier
folder_id	INT	FK folders.id, NULLABLE	Si partage dossier
type	ENUM('file','folder')	NOT NULL	Type de partage
password	VARCHAR(255)	NULLABLE	Hash si protégé
expires_at	DATETIME	NULLABLE	Date d'expiration
created_at	DATETIME	NOT NULL	Date de création

Contrainte : `CHECK (file_id IS NOT NULL OR folder_id IS NOT NULL)`

4.2.6 Table shared_folders

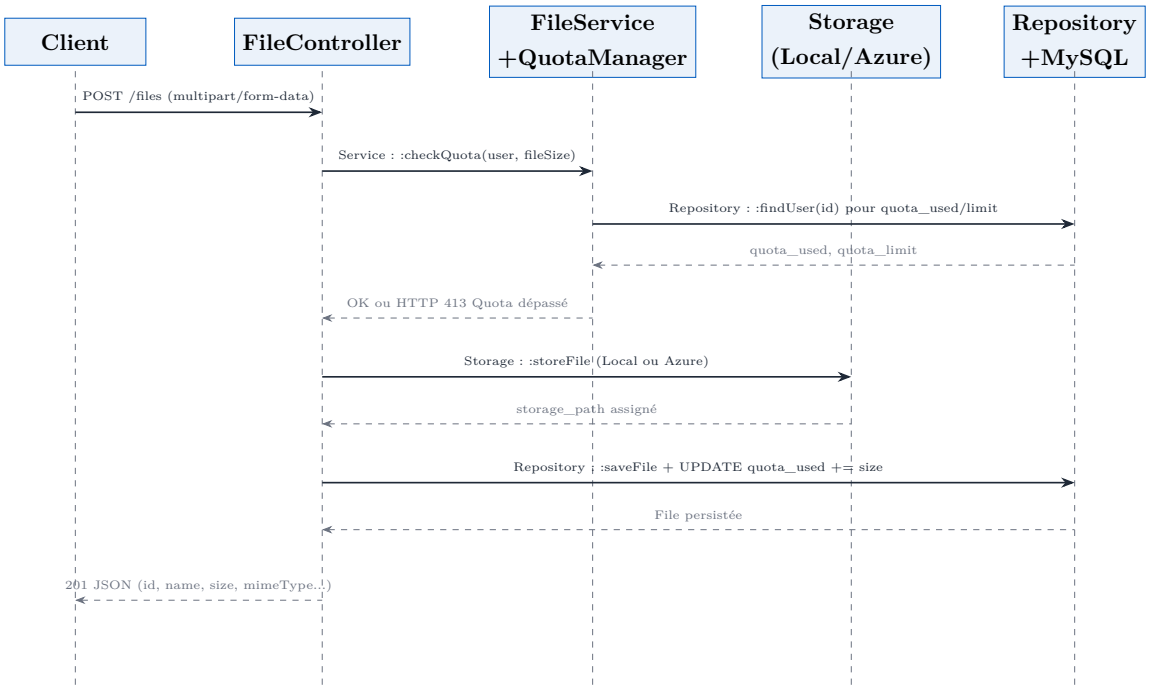
TABLE 4.5: Structure de la table `shared_folders` — partage privé

Colonne	Type	Contraintes	Description
id	INT	PK, AUTO_INCREMENT	Identifiant unique
folder_id	INT	FK folders.id, NOT NULL	Dossier partagé
shared_with_user	INT	FK users.id, NOT NULL	Destinataire
permission	ENUM('read','write')	DEFAULT 'read'	Type d'accès
created_at	DATETIME	NOT NULL	Date de partage

Contrainte : UNIQUE sur `(folder_id, shared_with_user)` — empêche les doublons de partage.

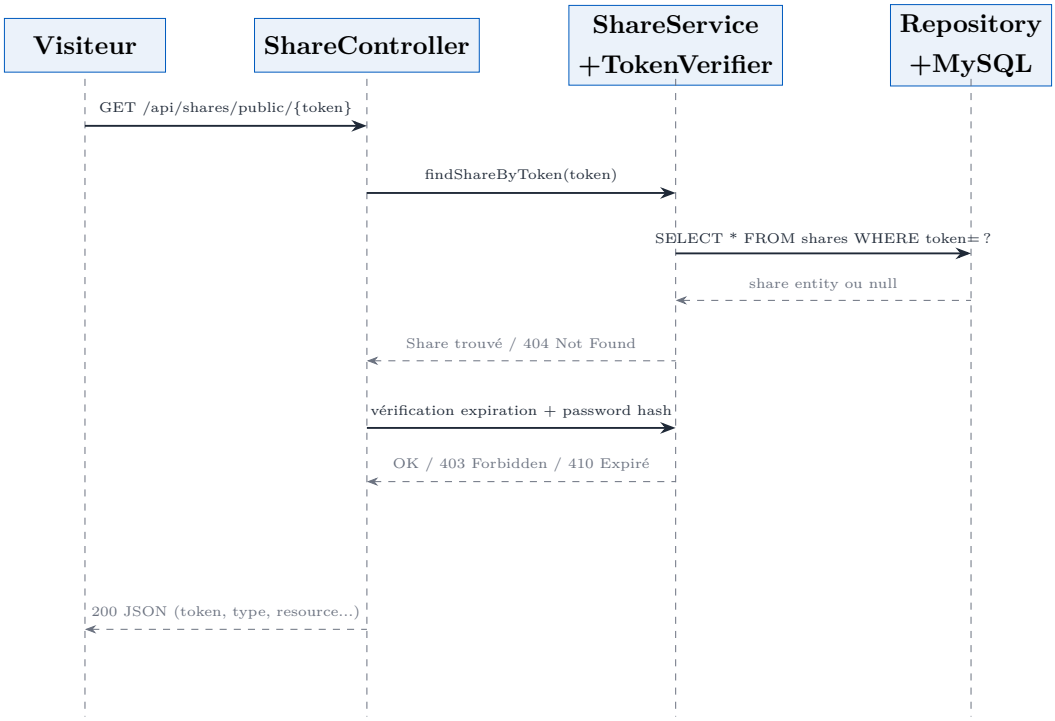
4.3 Diagramme de Séquence — Upload de Fichier

Ce diagramme illustre le flux complet d'un upload : vérification du quota, stockage physique, persistance des métadonnées.



4.4 Diagramme de Séquence — Accès à un Partage Public

Ce diagramme illustre le flux d'accès à un lien de partage par un visiteur (non authentifié).



Chapitre 5

Architecture de l'API REST

5.1 Conventions Générales

— **Base URL :** http ://192.168.0.22 :8000/api

— **Format :** JSON (`Content-Type: application/json`)

— **Authentification :** `Authorization: Bearer <JWT>` (sauf routes publiques)

— **Codes :** 200 OK, 201 Created, 204 No Content, 400, 401, 403, 404, 409, 422, 429 (rate limit)

5.2 Codes d'Erreur API

Code	Cas typique	Description
200	Requête réussie	Corps JSON avec la ressource demandée
201	Création réussie	Upload fichier, création dossier
204	Suppression réussie	Pas de corps de réponse
400	Données invalides	Champ manquant, format incorrect
401	Non authentifié	JWT absent, expiré ou invalide
403	Accès refusé	Voter Symfony : ressource d'un autre utilisateur
404	Non trouvé	Ressource inexistante ou soft-deleted
409	Conflit	Nom de dossier déjà utilisé
413	Fichier trop grand	Dépasse 2 Go ou quota utilisé
422	Validation échouée	Contraintes Symfony non respectées
429	Rate limit atteint	Trop de requêtes (forgot-password)

5.3 Authentification (`/api/auth`)

5.3.1 Principe JWT

L'API utilise l'authentification JWT (JSON Web Token) avec des clés RSA 4096 bits. Le processus est le suivant :

1. L'utilisateur envoie ses identifiants via `POST /api/auth/login`
2. Le serveur vérifie les identifiants et génère un JWT signé RS256
3. Le token a une durée de vie de **1 heure** (3600 secondes)
4. Le client inclut ce token dans toutes les requêtes via `Authorization: Bearer {token}`
5. Le serveur vérifie la validité du token avec la clé publique RSA

5.3.2 Niveaux de sécurité

SUPFile implémente 3 niveaux de sécurité :

1. **Firewall** (`security.yaml`) : détermine si une route nécessite un JWT valide
2. **Access Control** (`security.yaml`) : vérifie le rôle (`ROLE_USER`)
3. **Voters** (`FileVoter`, `FolderVoter`) : vérifie que l'utilisateur est propriétaire de la ressource (protection IDOR)

Méthode	Endpoint	Auth	Description
POST	`/auth/register`	Non	Inscription (nom, e-mail, mdp)
POST	`/auth/login`	Non	Connexion → JWT
POST	`/auth/forgot-password`	Non	E-mail reset (rate-limited)
POST	`/auth/reset-password`	Non	Reset mot de passe via token

Rate limiting — forgot-password

Pour des raisons de sécurité, la route `/auth/forgot-password` est soumise à un rate limiting strict : **3 demandes toutes les 15 minutes** par adresse IP. En cas de dépassement, le serveur retourne HTTP 429 *Too Many Requests*. Après 15 minutes, le compteur est réinitialisé.

5.3.3 Routes OAuth2

Méthode	Route	Auth	Description
GET	`/auth/oauth/google`	Non	Démarre OAuth Google
GET	`/auth/oauth/google/check`	Non	Callback web
GET	`/auth/oauth/google/check-json`	Non	Callback mobile (JSON)
GET	`/auth/oauth/github`	Non	Démarre OAuth GitHub
GET	`/auth/oauth/github/check`	Non	Callback web
GET	`/auth/oauth/github/check-json`	Non	Callback mobile (JSON)
GET	`/auth/oauth/microsoft`	Non	Démarre OAuth Microsoft
GET	`/auth/oauth/microsoft/check`	Non	Callback web
GET	`/auth/oauth/microsoft/check-json`	Non	Callback mobile (JSON)

Lorsqu'un utilisateur se connecte via OAuth :

- Si un compte OAuth existe déjà (même provider + oauthId) : connexion directe.
- Si un compte existe avec le même email : liaison automatique du compte OAuth.
- Sinon : création d'un nouveau compte avec mot de passe aléatoire.

Après authentification OAuth, le serveur génère un token JWT standard, identique à celui obtenu via `/api/auth/login`.

5.4 Gestion des Fichiers (`/api/files`)

5.4.1 Concepts clés

- **Quota** : Chaque utilisateur dispose de 30 Go (32 212 254 720 octets). Le quota est vérifié avant chaque upload.
- **Soft Delete** : Les fichiers supprimés sont marqués comme tels et déplacés dans la corbeille. Ils continuent de compter dans le quota jusqu'à suppression définitive.
- **Stockage** : Les fichiers sont stockés dans `backend/storage/{userId}/{uniqid}_{filename}`.

Méthode	Endpoint	Auth	Description
GET	`/files`	Oui	Lister (filtres, pagination)
POST	`/files`	Oui	Upload (multipart/form-data, max 2 Go)
GET	`/files/{id}/info`	Oui	Métadonnées d'un fichier
GET	`/files/{id}`	Oui	Télécharger le fichier
GET	`/files/{id}/preview`	Oui	Prévisualiser (stream)
PATCH	`/files/{id}`	Oui	Renommer (`{name}`)
PATCH	`/files/{id}/move`	Oui	Déplacer (`{folderId}`)
DELETE	`/files/{id}`	Oui	Supprimer (soft delete)

5.5 Gestion des Dossiers (`/api/folders`)

5.5.1 Hiérarchie

Les dossiers supportent une imbrication infinie (auto-référence via `parent`). La navigation utilise le paramètre `parentId` et le fil d'Ariane (breadcrumbs).

Méthode	Endpoint	Auth	Description
GET	`/folders`	Oui	Lister dossiers racine
POST	`/folders`	Oui	Créer un dossier
GET	`/folders/{id}`	Oui	Contenu d'un dossier
PATCH	`/folders/{id}`	Oui	Renommer (`{name}`)
PATCH	`/folders/{id}/move`	Oui	Déplacer (`{targetParentId}`)
DELETE	`/folders/{id}`	Oui	Supprimer (soft delete)
GET	`/folders/{id}/download`	Oui	Télécharger en ZIP
GET	`/folders/{id}/breadcrumbs`	Oui	Fil d'Ariane

5.6 Corbeille (`/api/trash`)

Les fichiers et dossiers supprimés passent d'abord en Corbeille (soft delete : `is_deleted=true`, `deleted_at=now()`). Ils sont masqués dans la navigation normale mais restent en base et sur le stockage. L'utilisateur peut les restaurer ou les supprimer définitivement (nettoyage du stockage physique + mise à jour de `quota_used`).

Méthode	Endpoint	Auth	Description
GET	`/trash`	Oui	Lister les éléments
POST	`/trash/files/{id}/restore`	Oui	Restaurer un fichier
POST	`/trash/folders/{id}/restore`	Oui	Restaurer un dossier
DELETE	`/trash/files/{id}`	Oui	Supprimer définitivement
DELETE	`/trash/folders/{id}`	Oui	Supprimer définitivement

5.7 Partage (`/api/shares`)

5.7.1 Types de partage

SUPFile supporte deux types de partage :

— **Partage public** : Génère un lien unique accessible sans authentification. Optionnellement

protégé par mot de passe et/ou date d'expiration.

— **Partage privé** : Partage avec un utilisateur spécifique identifié par son email. Nécessite que le destinataire ait un compte SUPFile. Le dossier partagé apparaît automatiquement dans la racine du destinataire.

Méthode	Endpoint	Auth	Description
POST	`/shares`	Oui	Créer un lien
GET	`/shares`	Oui	Mes partages actifs
DELETE	`/shares/{id}`	Oui	Révoquer
GET	`/shares/public/{token}`	Non	Infos du partage
POST	`/shares/public/{token}/validate`	Non	Valider mdp
POST	`/shares/public/{token}/download`	Non	Télécharger
GET	`/shares/public/{token}/preview`	Non	Prévisualiser

5.8 Tableau de Bord (`/api/dashboard`)

Le dashboard fournit des statistiques et une vue d'ensemble de l'espace de stockage.

Méthode	Endpoint	Auth	Description
GET	`/dashboard/stats`	Oui	Stats globales + quota
GET	`/dashboard/stats/type`	Oui	Répartition par type MIME
GET	`/dashboard/recent`	Oui	Fichiers récents
GET	`/dashboard/search`	Oui	Recherche globale

5.9 Gestion du Compte (`/api/user`)

Méthode	Endpoint	Auth	Description
GET	`/user/me`	Oui	Profil courant
PATCH	`/user/me`	Oui	Modifier nom, prénom
POST	`/user/me/password`	Oui	Changer le mot de passe
POST	`/user/me/avatar`	Oui	Upload avatar (max 2 Mo)
DELETE	`/user/me`	Oui	Supprimer le compte

5.10 Pages Légales (`/api/legal`) — Accès Public

Les trois pages légales sont servies en Markdown par le backend Symfony et accessibles sans authentification. Elles sont utilisées par le frontend web et mobile pour afficher les documents lors de l'inscription et en pied de page.

Méthode	Endpoint	Auth	Description
GET	`/legal/terms`	Non	CGU (Markdown)
GET	`/legal/privacy`	Non	RGPD (Markdown)
GET	`/legal/help`	Non	Centre d'aide

Sources Markdown

Les contenus légaux sont rédigés dans `backend/legal/` (`terms.md`, `privacy.md`, `help.md`). Le contrôleur Symfony lit ces fichiers et les retourne via un endpoint séparé du firewall privé — la route est déclarée `stateless: true` et accessible sans JWT (`pattern: ^/api/(public|legal)`).

Chapitre 6

Sécurité de l'Application

6.1 Protection contre les Injections SQL

Aucune requête SQL brute n'est exécutée. Toutes les requêtes passent par le **QueryBuilder Doctrine** qui génère exclusivement des *prepared statements* :

```
return $this->createQueryBuilder('f')
    ->where('f.owner = :owner')
    ->andWhere('f.name LIKE :search')
    ->setParameter('owner', $owner)
    ->setParameter('search', '%' . $search . '%')
    ->getQuery()->getResult();
```

Listing 6.1 – FileRepository::searchByName() — QueryBuilder securise

6.2 Protection Path Traversal

Les chemins physiques sont produits par `FileStorageInterface::storeFile` puis enregistrés dans l'entité `File` via `setPath` (`FileManager`) : ils ne sont pas construits par concaténation directe d'un chemin arbitraire fourni par le client. Le code applicatif vit sous `backend/src/Service/File/` et `backend/src/Service/Storage/`.

6.3 Rate Limiting — Mot de Passe Oublié

— Maximum 5 tentatives / heure par adresse IP

— Maximum 3 tentatives / heure par adresse email

— Retour HTTP 429 *Too Many Requests* en cas de dépassement

Valeur unifiée du rate limit

La valeur officielle est **3 demandes / 15 min par IP**, définie dans `backend/config/packages/rate_limiter.yaml`. Cette valeur est la référence unique dans tout le projet (code, documentation, Manuel Utilisateur).

6.4 Sécurité Mobile — expo-secure-store

— **iOS :** Keychain Services — chiffrement AES-256 (Secure Enclave)

— **Android :** Android Keystore System — chiffrement matériel

— Jamais en `AsyncStorage` (stockage en clair, non conforme MASVS-STORAGE-1)

6.5 OAuth 2.0 — WebView Isolée (Mobile)

— `incognito=true`, `domStorageEnabled=false`, `thirdPartyCookiesEnabled=false`

— User-Agent Chrome Mobile standard (contourne la détection WebView Google)

— Guard `handledRef` empêchant les doubles callbacks

— Extraction du token depuis `/auth/callback?token=JWT`

6.6 Conformité OWASP Top 10

Risque OWASP	Statut	Mesure implémentée
A01 Broken Access Control	OK	Voters Symfony, ownership check systématique
A02 Cryptographic Failures	OK	JWT RS256, bcrypt/argon2, HTTPS Cloudflare
A03 Injection	OK	Doctrine ORM uniquement, 0 SQL brut
A04 Insecure Design	OK	Security by Default via start.sh
A05 Security Misconfiguration	OK	.env non commité, Docker secrets
A07 Auth Failures	OK	Rate limiting, JWT TTL 1h, token à usage unique
A08 Data Integrity	OK	Validation Symfony, types stricts PHP 8.3

Chapitre 7

Justification des Choix Technologiques

7.1 Backend : Symfony 7.4 LTS / PHP 8.3

Critère	Choix	Justification
Framework	Symfony 7.4 LTS	Pattern MVC,facilité de maintenance et d'évolution
ORM	Doctrine 3	QueryBuilder sécurisé, migrations versionées
Auth	LexikJWT + KnpU	JWT RS256, OAuth 2.0 PKCE complet
RBAC	Symfony Voters	Contrôle granulaire par ressource
Rate limit	RateLimiter	Protection brute-force, forgot-password

Symfony a été retenu face à Node.js en raison de son architecture structurée (Controllers, Services, Repositories), de sa gestion native de la sécurité, de la richesse de son écosystème de bundles (Doctrine, LexikJWT, etc.) et de la maturité de sa documentation.

7.2 Frontend Web : React 19 + TypeScript + Vite

Bibliothèque	Version	Rôle
React	19	UI déclarative, composants réutilisables
TypeScript	5.9+	Typage strict, détection d'erreurs à la compilation
Vite	7	Bundler ultra-rapide, HMR instantané
TanStack Query	5	Cache serveur, invalidation automatique
Zustand	5	État global minimal (auth, UI)
Axios	1.x	Client HTTP avec intercepteurs JWT
React Router	7	Routing SPA côté client

Le JWT est stocké exclusivement en `sessionStorage` (jamais `localStorage`) : vidé à la fermeture d'onglet, fenêtre d'attaque XSS réduite. Tout token résiduel en `localStorage` est migré et supprimé au chargement.

7.3 Frontend Mobile : React Native + Expo SDK 54

Outil	Version	Rôle
React Native	0.81	Framework cross-platform iOS/Android
Expo SDK	54	Accès APIs natives simplifié
expo-secure-store	15	Stockage chiffré JWT (Keychain/Keystore)
expo-av	16	Lecture audio/vidéo native
expo-document-picker	14	Sélection de fichiers système
expo-image-picker	17	Accès galerie photos
react-native-webview	13	OAuth 2.0 via navigateur intégré
EAS Build	12	Build APK/IPA en cloud

Le frontend web repose sur React 19, TypeScript et Vite plutôt que Vue.js ou Angular, ces derniers ayant été écartés respectivement pour leur moindre disponibilité de ressources, leur lourdeur inadaptée à l'échelle du projet.
Pour le mobile, React Native et Expo SDK 54 ont été préférés à Flutter, dont le langage Dart représente une courbe d'apprentissage supplémentaire, à Ionic, pénalisé par ses performances WebView, et au développement natif Swift/Kotlin, jugé trop coûteux en temps face à une cible cross-platform iOS/Android.

Chapitre 8

Guide de Déploiement

8.1 Déploiement Local (Docker Compose) — Architecture Principale

Le déploiement local avec Docker Compose est l'**architecture de référence** du projet. Un simple `./start.sh` lance l'intégralité de la plateforme (backend, base de données, frontend web) en quelques secondes. L'application mobile s'exécute hors Docker via Expo.

Mobile non dockerisé — Connecté à la production par défaut

Le frontend mobile (React Native + Expo SDK 54) est exclu du `docker-compose.yml`. Il se lance via `npx expo start` et communique avec l'API via la variable d'environnement `EXPO_PUBLIC_API_URL` définie dans `frontend-mobile/.env`.

Configuration actuelle (production) :

```
# frontend-mobile/.env
EXPO_PUBLIC_API_URL=https://api.supfile-cloud.fr/api
```

Pour basculer en développement local :

1. S'assurer que les conteneurs Docker sont en cours d'exécution (`make start`)
2. Trouver l'adresse IP LAN de la machine (ex. : `ifconfig | grep "inet "` sur macOS)
3. Modifier `frontend-mobile/.env` :

```
# frontend-mobile/.env --- mode developpement local
EXPO_PUBLIC_API_URL=http://192.168.x.x:8000/api
# Remplacer 192.168.x.x par votre IP LAN reelle
```

Pourquoi l'IP LAN et pas localhost ?

Le simulateur/téléphone physique s'exécute dans un réseau différent de la machine hôte. `localhost` réfère à l'appareil lui-même, pas à votre ordinateur. L'IP LAN (ex. : `192.168.1.x`) est l'adresse accessible depuis l'appareil sur le même réseau Wi-Fi.

8.2 Déploiement Prod : Microsoft Azure — Bonus

En complément du déploiement local, le projet a été mis en production sur Microsoft Azure pour valider la scalabilité de l'architecture. Le pattern `StorageFactory` (variable `STORAGE_DRIVER=azure`) bascule automatiquement vers Azure Blob pour les fichiers et Azure MySQL pour les métadonnées.

Infrastructure de Production Azure

— **App Service** B1 — Italy North (backend Symfony conteneurisé)
— **Azure MySQL Flexible Server** B1ms — Poland Central (métadonnées)
— **Azure Blob Storage** LRS Standard — Europe (fichiers physiques)
— **Azure Container Registry** Basic — Images Docker privées
— **Cloudflare** — DNS, SSL/TLS Full Strict, CDN, protection DDoS

8.2.1 Variables d'environnement de Production

Variable	Description
`APP_ENV`	`prod`
`APP_SECRET`	Secret cryptographique Symfony
`DATABASE_URL`	DSN MySQL (user :pass@host :3306/db)
`JWT_SECRET_KEY`	Clé privée RSA — signature JWT
`JWT_PUBLIC_KEY`	Clé publique RSA — vérification JWT
`JWT_PASSPHRASE`	Passphrase de la clé privée
`AZURE_STORAGE_ACCOUNT`	Compte Azure Blob Storage
`AZURE_STORAGE_KEY`	Clé d'accès Azure Blob
`AZURE_STORAGE_CONTAINER`	Conteneur Blob
`GOOGLE_CLIENT_ID/SECRET`	OAuth Google
`GITHUB_CLIENT_ID/SECRET`	OAuth GitHub
`MAILER_DSN`	SMTP (emails reset mot de passe)
`STORAGE_DRIVER`	`azure` (bascule AzureBlobStorage)

Chapitre 9

Pipeline CI/CD

9.1 Vue d'Ensemble

Le pipeline GitHub Actions se déclenche sur chaque push ou pull request vers `main` uniquement. Le déploiement Azure n'est exécuté que sur les push directs vers `main`. Il comprend 6 jobs :

Job 1 — test-backend (PHPUnit)

Ce job lance les tests unitaires et d'intégration du backend Symfony sur une machine Ubuntu. Il démarre un service MySQL 8.0.45 pour simuler la base de données de test, installe les dépendances PHP via Composer, génère les clés JWT, chauffe le cache Symfony, applique les migrations Doctrine, puis exécute la suite de tests PHPUnit.

Job 2 — test-frontend (Jest)

Ce job exécute les tests du frontend web sur Ubuntu avec Node.js 22. Il installe les dépendances npm dans le dossier `frontend-web` puis lance Jest en mode CI.

Job 3 — security (audit + Trivy)

Ce job s'exécute uniquement après la réussite des deux jobs précédents. Il réalise un audit des dépendances PHP avec `composer audit`, un audit des dépendances JavaScript avec `npm audit` en bloquant sur les vulnérabilités de niveau élevé, puis effectue un scan Trivy sur les sources PHP et JS pour détecter les vulnérabilités critiques et hautes.

Jobs 5 & 6 — deploy-backend / deploy-frontend

Ces deux jobs de déploiement s'exécutent uniquement sur un push sur la branche `main`, après la réussite du job de build. Ils se connectent à l'Azure Container Registry, construisent l'image Docker de production, effectuent un scan Trivy sur l'image en bloquant sur toute vulnérabilité critique, poussent l'image taguée avec le SHA du commit, puis déploient l'application sur Azure Web Apps.

Secrets GitHub Actions requis

- `AZURE_CREDENTIALS` — Service principal Azure (JSON)
- `ACR_LOGIN_SERVER` — URL du registre de conteneurs Azure
- `ACR_USERNAME` / `ACR_PASSWORD` — Identifiants ACR
- `AZURE_WEBAPP_NAME_API` — Nom de l'App Service backend
- `AZURE_WEBAPP_NAME_WEB` — Nom de l'App Service frontend
- `JWT_SECRET_KEY` / `JWT_PUBLIC_KEY` — Pour les tests backend

Chapitre 10

Dépannage Technique (Troubleshooting)

10.1 Problèmes Courants et Solutions

10.1.1 Le conteneur `api` ne démarre pas

Symptôme

`docker-compose up` affiche des erreurs sur le service `api` ou le conteneur redémarre en boucle.

Causes et solutions :

1. **Fichier `.env` manquant** : vérifiez que `start.sh` a été exécuté. Relancez `./start.sh`.
2. **Clés JWT absentes** : vérifiez que `backend/config/jwt/private.pem` et `public.pem` existent. Relancez : `php bin/console lexik:jwt:generate-keypair` depuis le conteneur `api`.
3. **Port 8000 déjà occupé** : `lsof -i :8000` puis tuer le processus ou changer le port dans `docker-compose.yml`.
4. **Consultez les logs** : `make logs SERVICE=api`

10.1.2 Erreur de connexion MySQL au démarrage

Symptôme

`SQLSTATE[HY000] [2002] Connection refused` ou `Access denied for user`.

Solutions :

1. Le conteneur `database` peut prendre 10–20 secondes à démarrer. Attendez puis relancez : `make restart`.
2. Vérifiez les variables `DB_USER`, `DB_PASSWORD`, `DB_NAME` dans `.env`.
3. Logs MySQL : `make logs SERVICE=database`
4. En dernier recours : `docker volume rm supfile-project_db_data` puis `make rebuild`.

10.1.3 Migrations Doctrine échouent

Solutions :

1. Vérifier que la base existe : `php bin/console doctrine:database:create`
2. Lister les migrations en attente : `php bin/console doctrine:migrations:status`
3. Forcer l'exécution : `php bin/console doctrine:migrations:migrate -no-interaction`

10.1.4 Le mobile ne peut pas joindre l'API locale

Symptôme

L'application affiche `Network request failed` ou `ERR_CONNECTION_REFUSED`.

Solutions :

1. **Ne jamais utiliser `localhost`** : utilisez l'IP LAN (ex. `192.168.1.x`).
2. Trouver votre IP : `ifconfig | grep "inet "` (macOS/Linux) ou `ipconfig` (Windows).
3. Vérifier que le téléphone et l'ordinateur sont sur le **même réseau Wi-Fi**.
4. Vérifier que le port `8000` n'est pas bloqué par le pare-feu de la machine.
5. Modifier `frontend-mobile/.env` : `EXPO_PUBLIC_API_URL=http://192.168.x.x:8000/api`

10.1.5 Erreur OAuth — Callback refusé

Solutions :

1. Vérifier que l'URL de callback est enregistrée dans la console OAuth du fournisseur (Google, GitHub, Microsoft).
2. Vérifier `GOOGLE_CLIENT_ID`, `GITHUB_CLIENT_ID`, `MICROSOFT_CLIENT_ID` dans `.env`.
3. En local, l'URL de callback doit inclure le port : `http://localhost:8000/api/auth/oauth/google/`

10.1.6 Upload refusé — HTTP 413 ou quota dépassé

1. **HTTP 413 Payload Too Large** : le fichier dépasse 2 Go. Vérifier `client_max_body_size` dans Nginx et `upload_max_filesize` dans PHP.
2. **Quota dépassé** : l'utilisateur a atteint 30 Go. Supprimer des fichiers et vider la Corbeille pour libérer de l'espace.

10.1.7 JWT expiré — HTTP 401 après 1 heure

Le token JWT a une durée de vie de **1 heure**. Après expiration :

1. L'API retourne `401 Unauthorized` avec le message `"Expired JWT Token"`.
2. Le client doit redemander un token via `POST /api/auth/login`.
3. Sur le frontend web, l'intercepteur Axios gère automatiquement la redirection vers la page de connexion.

10.1.8 Consulter les Logs

— `make logs` — logs en temps réel de tous les services
— `make logs SERVICE=api` — logs du backend uniquement
— `make logs SERVICE=database` — logs MySQL
— `docker exec -it supfile-api php bin/console debug:router` — lister toutes les routes Symfony
— `docker exec -it supfile-api php bin/console debug:container` — inspecter les services Symfony

Chapitre 11

Charte Graphique

11.1 Palette de Couleurs

Couleur	Code hex	Usage
	`#005BBF`	Couleur primaire — boutons, liens, accents
	`#1A73E8`	Primaire clair — gradient, hover
	`#1F2937`	Dark gray — titres, texte principal
	`#F3F4F6`	Light gray — fonds, sidebar
	`#E5E7EB`	Border — séparateurs, bordures
	`#059669`	Succès — confirmations, badges OK
	`#DC2626`	Erreur — alertes, actions destructives
	`#D97706`	Avertissement — messages de prudence

11.2 Gradient Principal

```
background: linear-gradient(135deg, #005BBF 0%, #1A73E8 100%);
```

Listing 11.1 – CSS — Gradient de marque

Utilisé sur les en-têtes, boutons primaires et éléments d'illustration.

11.3 Typographie

Élément	Police	Spécifications
Corps du texte	Inter	16px, weight 400, line-height 1.6
Titres H1	Inter	32px, weight 700, color `#1F2937`
Titres H2	Inter	24px, weight 600, color `#1F2937`
Boutons	Inter	15px, weight 600, letter-spacing 0.01em
Code / mono	JetBrains Mono	14px, weight 400

Inter est importée depuis Google Fonts, sous-ensemble latin, variantes 300–700.

11.4 Thèmes Clair et Sombre

```
:root {
  --bg-primary: #FFFFFF; --text-primary: #1F2937;
  --bg-secondary: #F3F4F6; --text-secondary: #6B7280;
  --border-color: #E5E7EB; --primary: #005BBF;
```

```
}
[data-theme="dark"] {
  --bg-primary: #111827; --text-primary: #F9FAFB;
  --bg-secondary: #1F2937; --text-secondary: #9CA3AF;
  --border-color: #374151; --primary: #3B82F6;
}
```

Listing 11.2 – Variables CSS — theming

11.5 Règles de Design

— **Coins arrondis :** `8px` (petits), `12px` (cartes), `50%` (avatars)

— **Ombres :** `box-shadow: 0 1px 3px rgba(0,0,0,0.1)` — légères

— **Transitions :** `transition: all 0.2s ease`

— **Icônes :** Ionicons (mobile) / Lucide React (web)

— **Espacement :** Grille de 4px (4, 8, 12, 16, 24, 32, 48, 64px)

Chapitre 12

Fonctionnalités et Qualité du Code

12.1 Fonctionnalités Implémentées

Fonctionnalité	Backend	Web	Mobile
Inscription / Connexion JWT	OK	OK	OK
OAuth Google / GitHub / Microsoft	OK	OK	OK
Mot de passe oublié	OK	OK	OK
Upload fichiers (max 2 Go)	OK	OK	OK
Téléchargement	OK	OK	OK
Prévisualisation image/PDF	OK	OK	OK
Lecture vidéo / audio	OK	OK	OK
Gestion dossiers	OK	OK	OK
Renommer / Déplacer	OK	OK	OK
Partage lien public	OK	OK	OK
Partage protégé par mdp	OK	OK	OK
Partage avec expiration	OK	OK	OK
Partage privé entre users	OK	OK	OK
Tableau de bord analytique	OK	OK	OK
Recherche globale	OK	OK	OK
Gestion du compte	OK	OK	OK
Corbeille (soft delete + restore)	OK	OK	OK
Quota utilisateur 30 Go	OK	OK	OK
Téléchargement ZIP dossier	OK	OK	OK
Thème sombre	N/A	OK	OK
Stockage Azure Blob (prod, bonus)	OK	N/A	N/A

12.2 Standards de Qualité

Couche	Standard	Outil
Backend PHP	PHPUnit 12	`phpunit/phpunit` en `require-dev`
Backend PHP	Lint Symfony	`lint:yaml`, `lint:twig` (cible `make lint`)
Frontend Web	ESLint	ESLint + Prettier
Frontend Mobile	ESLint	ESLint (config Expo)
TypeScript	Strict mode	`-strict` TypeScript 5.9

— **SOLID :** Contrôleurs délèguent au Service, Repositories encapsulent les données, Voters implémentent OCP.

— **DRY :** Pattern StorageFactory évite la duplication de logique local/Azure.

— **Typage PHP :** le projet cible PHP 8.3 dans Docker ; `composer.json` impose `php >= 8.2`. Les fichiers sous `backend/src/` n'utilisent pas systématiquement `declare(strict_types=1);` (convention Symfony courante).

— **Cache serveur :** TanStack Query gère cache, revalidation et états de chargement/erreur.

12.3 Stratégie de Tests

La stratégie de tests du projet SUPFile couvre les trois couches de l'application : le backend Symfony, le frontend web React et le frontend mobile React Native. Chaque couche dispose de tests unitaires et, selon la pertinence, de tests d'intégration.

12.3.1 Backend — Tests PHP (PHPUnit 12)

Le backend est testé avec **PHPUnit 12** (`phpunit/phpunit` en `require-dev` du `backend/composer.json`). Les tests sont situés dans le répertoire `backend/tests/` et sont exécutés automatiquement dans le pipeline CI/CD.

Suite	Type	Périmètre
`AuthControllerTest`	Intégration	Inscription, login JWT, validation, routes protégées
`FileControllerTest`	Intégration	Upload, téléchargement, suppression, renommage
`FolderControllerTest`	Intégration	Création, déplacement, ZIP
`ShareControllerTest`	Intégration	Partage public, privé, mdp, expiration
`TrashControllerTest`	Intégration	Soft-delete, restauration, suppression d'efinitive
`UserControllerTest`	Intégration	Profil, mot de passe, avatar
`DashboardControllerTest`	Intégration	Stats, recherche globale
`LocalFileStorageTest`	Unitaire	Driver de stockage local

— **Base de données :** les tests d'intégration s'exécutent contre **MySQL** (service MySQL dans GitHub Actions ; en local, base `supfile_db_test` préparée via `make test-setup` puis conteneur `api`).

— **Cas de tests clés :** validation de la limite de taille d'avatar (2 Mo → HTTP 400), vérification des droits d'accès entre utilisateurs, écriture et lecture de fichier via le driver local.

— **Exécution :** `php bin/phpunit -testdox` depuis le répertoire `backend/`.

— **CI/CD :** les tests sont exécutés automatiquement au job `test-backend` du pipeline GitHub Actions.

12.3.2 Frontend Web — Tests TypeScript (Jest)

Le frontend web est testé avec **Jest** et **Testing Library React**. Les tests sont organisés en deux niveaux dans le répertoire `frontend-web/src/tests/`.

Suite	Type	Périmètre
`unit/services/authService`	Unitaire	Appels API auth (register, login, etc.)
`unit/services/fileService`	Unitaire	Service fichiers / dossiers
`unit/authStore`	Unitaire	Store Zustand auth
`integration/components/LoginForm`	Intégration	Formulaire login, validation
`integration/hooks/useFiles`	Intégration	Hook fichiers (chargement, erreurs)
`integration/hooks/useShares`	Intégration	Hook partages

— **Mocking :** Jest (`jest.spyOn`, modules mockés dans `src/tests/setup.ts` / tests) pour isoler services et hooks.

— **Couverture :** `npx jest -coverage` (voir cible `make test-frontend-web-coverage`).

— **Exécution :** `npx jest` (local : `make test-frontend-web`) ou `npx jest -ci` (GitHub Actions `test-frontend`) depuis `frontend-web/`.

— **CI/CD :** exécutés au job `test-frontend` du pipeline GitHub Actions.

12.3.3 Frontend Mobile — Tests TypeScript (Jest + Testing Library)

Le frontend mobile est testé avec **Jest** (préconfiguré par Expo) et **Testing Library React Native**. Les tests sont situés dans `frontend-mobile/src/tests/` et se décomposent en deux catégories.

Tests Unitaires (`src/tests/unit/`)

Fichier de test	Périmètre couvert
`auth.test.ts`	Endpoints d'authentification (register, login, getMe, token)
`authStore.test.ts`	Store Zustand : login, logout, checkAuth, setOAuthSession
`client.test.ts`	Intercepteurs Axios : ajout du header JWT, gestion des 401
`dashboard.test.ts`	Appels API getStats, getStatsByType, getRecentFiles
`fileHelpers.test.ts`	Fonctions utilitaires : formatFileSize, formatDate, getMimeIcon
`files.test.ts`	CRUD fichiers/dossiers, upload avec progression
`filesStore.test.ts`	Store Zustand : setFiles, setFolders, clearCache
`profile.test.ts`	API profil : updateProfile, changePassword, uploadAvatar
`shares.test.ts`	API partages : création, révocation, accès public
`syncStore.test.ts`	Store de synchronisation : triggerRefresh
`themeStore.test.ts`	Store thème : loadTheme, setTheme, toggleTheme
`trash.test.ts`	API corbeille : getTrash, restore, deleteFile/Folder
`useTheme.test.ts`	Hook useTheme : retourne la bonne palette clair/sombre
`useToast.test.ts`	Hook useToast : showToast, hideToast, minuteur automatique

Tests d'Intégration (`src/tests/integration/components/`)

Fichier de test	Périmètre couvert
`LoginScreen.test.tsx`	Rendu du formulaire, validation des champs vides, navigation
`RegisterScreen.test.tsx`	Champs requis, consentement CGU obligatoire, appel register()
`DashboardScreen.test.tsx`	Chargement des statistiques, affichage du prénom
`FilesScreen.test.tsx`	Appels API au montage, rendu avec dossier courant
`ShareScreen.test.tsx`	Onglets Mes partages / Reçus, appel API de chargement
`ProfileScreen.test.tsx`	Affichage du nom et email de l'utilisateur connecté
`TrashScreen.test.tsx`	Message corbeille vide, appel trashApi.getTrash
`Toast.test.tsx`	Rendu selon les types success, error, info
`OAuthWebViewModal.test.tsx`	Visibilité de la modal (visible / cachée)

Chapitre 13

Conclusion

13.1 Bilan

SUPFile est une plateforme de stockage cloud complète et fonctionnelle, déployée en production sur Azure et accessible sur `https://www.supfile-cloud.fr`. Toutes les fonctionnalités du cahier des charges sont implémentées sur les trois couches.

13.2 Points Forts Techniques

— **Sécurité by design :** JWT RS256, bcrypt, Voters RBAC, rate limiting, secrets générés cryptographiquement

— **Portabilité :** Docker Compose, un seul script `./start.sh`

— **Scalabilité :** Pattern Storage abstrait local → Azure sans modification du code métier

— **CI/CD :** Pipeline 6-jobs : `test-backend` (PHPUnit), `test-frontend` (Jest), audit sécurité, build frontend, déploiement Docker backend et frontend sur Azure

— **Mobile sécurisé :** JWT en Keychain/Keystore, OAuth via WebView isolée

13.3 Perspectives d'Évolution

— Versionnement des fichiers (historique)

— Synchronisation offline mobile

— Notifications push (Expo Push Notifications)

— Administration back-office (EasyAdmin 4)

Références

— Symfony 7.4 : https://symfony.com/doc/7.4/index.html
— LexikJWT Bundle : https://github.com/lexik/LexikJWTAuthenticationBundle
— KnpU OAuth2 Bundle : https://github.com/knpuniversity/oauth2-client-bundle
— Doctrine ORM : https://www.doctrine-project.org/projects/orm.html
— Expo SDK 54 : https://docs.expo.dev/
— EAS Build : https://docs.expo.dev/build/introduction/
— TanStack Query v5 : https://tanstack.com/query/v5
— Azure App Service : https://docs.microsoft.com/fr-fr/azure/app-service/
— OWASP Top 10 : https://owasp.org/www-project-top-ten/
— MASVS : https://mas.owasp.org/MASVS/

Dépôt Git du Projet

https://github.com/Abdoulayefoff/supfile-project

Annexe — Processus de Développement

SUPFile a été développé selon une approche **hybride** : quatre phases globales séquentielles ont structuré la progression (cadrage, socle, fonctionnalités, finalisation), enrichies d'itérations internes permettant ajustements et revues de code sans gel du périmètre. Le backend API centralisé conditionnant les deux clients (web et mobile), un contrat d'interface commun a servi de point de synchronisation tout au long du projet.

1 Cadrage

8–31 déc. 2025

Analyse du cahier des charges, choix de la stack et répartition des rôles. Conception du modèle de données (5 tables) et mise en place du dépôt Git.

Période	Livrable
8–31 déc. 2025	Modèle de données, architecture Docker, répartition des rôles

2 Socle Technique

jan. 2026

Mise en place de Docker Compose, authentification JWT RS256 + OAuth 2.0, script `start.sh`. Définition du contrat d'API partagé avec les équipes frontend.

Période	Livrable
Janvier 2026	API auth, environnement Docker, contrat d'API

3 Fonctionnalités

fév.–mars 2026

Implémentation intégrale du cahier des charges : partage, corbeille, quota, ZIP, prévisualisation, tableau de bord. Frontend web et mobile complets. Sécurité RBAC (Voters) et rate limiting.

Période	Livrable
Fév. 2026	Backend complet : API fichiers, partage, corbeille, quota
Mars 2026	Frontend web React + application mobile React Native

4 Finalisation

avr.–mai 2026

Suites de tests complètes (PHPUnit, Jest, Testing Library), corrections d'incohérences, relecture sécurité OWASP/MASVS. Finalisation de la documentation technique et du Manuel Utilisateur.

Période	Livrable
Avril 2026	Tests backend, corrections d'incohérences
Mai 2026	Tests frontend/mobile, documentation technique v1.0

+ Bonus — Azure + CI/CD

mai 2026

Déploiement production sur Azure (App Service, MySQL Flexible, Blob Storage, Cloudflare). Pipeline GitHub Actions 6 jobs avec scan Trivy et déploiement Docker automatisé.

Période	Livrable
Mai 2026	Infrastructure Azure, pipeline CI/CD, https://www.supfile-cloud.fr

Dépôt Git du Projet

https://github.com/Abdoulayefoff/supfile-project